



AuthClaw

Engineering Project Plan & Delivery Blueprint

AI Compliance Gateway · Agentic Remediation Engine · Continuous Audit

Field	Detail
Document	AuthClaw — Software Project Plan (Build-to-MVP)
Source spec	AuthClaw Software Requirements Specification (SRS)
Prepared by	Binod Kumar — Founder & Chief AI Officer
Delivery brand	AgentsArchitects.ai
Audience	Engineering / Delivery Team
Version	v1.0 — Draft for team review
Total estimated effort	2,560 engineering hours (4 phases × 640 h)

<https://agentsarchitects.ai/>

Document Control

Item	Detail
Title	AuthClaw Engineering Project Plan
Owner	Binod Kumar
Status	Draft v1.0 for engineering review
Reference inputs	AuthClaw SRS; product IA benchmarked against Vanta, Securiti, Giskard
Estimation basis	Effort expressed in engineering hours per the SRS phase model
Distribution	Internal — delivery pod, tech lead, QA/security

Revision History

Ver.	Date	Author	Summary
1.0	Initial draft	Binod Kumar	First full plan derived from the SRS; epics, sprints, RACI, risk register, DoD.

Table of Contents

1. Executive Summary & Objectives

AuthClaw is a B2B SaaS platform that evaluates, monitors, and remediates AI applications against regulatory frameworks — GDPR, HIPAA, and SOC 2 Type II. This plan converts the approved SRS into an executable engineering blueprint: a phased breakdown of epics, tasks, owners, dependencies, sprint allocation, acceptance criteria, and a risk register that the delivery pod can start working against immediately.

The product is delivered across three coupled pillars, each becoming a delivery workstream:

- **In-Line Security Gateway (Proxy)** — a low-latency reverse proxy that intercepts traffic between client apps and foundation models, redacts PII/PHI, and enforces policy before egress.
- **Agentic Chat & Remediation Engine** — an Orchestrator–Worker system that scans client environments, explains compliance gaps via RAG, and executes approved fixes under Human-in-the-Loop (HITL) control.
- **Continuous Observability & Audit Trail** — a tamper-proof, append-only log of model I/O, agent reasoning, and approvals, exportable as cryptographically verifiable compliance evidence.

Delivery target

MVP scope = all three pillars at production quality across 4 phases of 640 engineering hours each (2,560 h total). Estimates are stated in engineering hours; an indicative calendar follows from the team-capacity assumption in Section 7.

1.1 Primary Objectives

1. Ship a multi-model proxy that adds no more than 50 ms processing overhead per request while redacting sensitive data in real time, including on streaming responses.
2. Deliver an agentic remediation engine where every destructive action is gated behind explicit, expiring HITL approval with MFA.
3. Provide automated framework scoring (SOC 2 / GDPR / HIPAA) and cryptographically verifiable audit export.
4. Enforce strict multi-tenant isolation and zero-trust security from day one, not as a hardening afterthought.

2. Scope

2.1 In Scope (MVP)

Area	Included in MVP
Gateway	Reverse proxy for OpenAI, Anthropic, Cohere, Azure OpenAI; PII/PHI redaction (mask / hash / synthetic); YAML + OPA policy enforcement; streaming-safe filtering.
Agentic engine	LangGraph orchestrator; ephemeral scoped workers; AWS / GCP / GitHub connectors; RAG over regulatory docs; HITL approval workflow with 0.5 h expiry and MFA gate.
Compliance & audit	Real-time framework scoring; ClickHouse immutable hash-chained audit log; cryptographically verified export; Trust Center page.

Area	Included in MVP
Console	Next.js admin console: dashboard, gateway config, policy editor, agent chat, approvals queue, audit explorer, tenant/RBAC/API-key admin.
Platform	Multi-tenant RBAC, OIDC/IdP auth, envelope-encrypted secrets, multi-region active-active, tiered rate limiting, CI/CD.

2.2 Out of Scope (MVP) — backlog candidates

- Additional frameworks beyond SOC 2 / GDPR / HIPAA (e.g. ISO 27001, ISO 42001, EU AI Act, PCI) — architected for, not built.
- Marketplace / partner directory, billing & metering beyond plan-tier stubs, and questionnaire automation.
- On-prem / air-gapped deployment packaging (cloud-hosted multi-tenant first).
- Non-text modalities (image/audio model proxying) and providers outside the four named above.

3. Product Navigation & Information Architecture

The console menu is structured around the SRS pillars and benchmarked against the reference products you flagged. Vanta informs the compliance-scoring, trust-center, and audit-export surfaces; Giskard informs the guardrails and red-teaming surfaces; Securiti informs the sensitive-data discovery and redaction surfaces. The proposed top-level navigation:

#	Menu item	Purpose	Backing modules
1	Overview	Tenant home: live SOC 2 / GDPR / HIPAA readiness %, open approvals, redaction & traffic KPIs.	FR-3.1, audit store
2	Gateway	Endpoint & model-routing config, redaction strategy per route, live traffic inspector.	Module 3.1 (FR-1.1–1.3)
3	Policies & Guardrails	YAML policy-as-code editor, OPA rules, guardrail taxonomy (prompt injection, data disclosure, sycophancy, harmful content).	FR-1.3, FR-2.x guardrails
4	Agent & Remediation	Conversational compliance assistant, scan results, remediation plans (Terraform/CLI diffs), HITL approvals queue.	Module 3.2 (FR-2.1–2.3)
5	Frameworks	Control-by-control mapping & scoring for SOC 2, GDPR, HIPAA; evidence linkage.	FR-3.1
6	Audit & Trust Center	Immutable log explorer, hash-chain verification, cryptographic export, shareable trust page.	FR-3.2, audit schema
7	Risk & Red Teaming	Continuous adversarial probe runs, vulnerability register, go/no-go posture.	Phase 4 red-team harness
8	Integrations	Cloud & SCM connectors (AWS, GCP, GitHub, Azure) and model-provider credentials.	Ephemeral workers
9	Settings	Tenants, RBAC, users, API keys, secret management, rate-limit tiers, billing tier.	NFR-2.x, FR-3.x

Reference mapping (so the team knows the 'good' bar)

Vanta → Overview scoring, Frameworks, Audit & Trust Center, agentic GRC assistant pattern.
Giskard → Policies & Guardrails taxonomy + Risk & Red Teaming (continuous probes, vulnerability severity,

go/no-go report).

Securiti → Gateway redaction & sensitive-data classification (Presidio + NER) and data-flow visibility.

4. Architecture & System Topology

Decoupled, microservices-based, built for high throughput and zero-trust security. Three layers map directly to delivery workstreams.

4.1 Layer 1 — In-Line Security Gateway (Proxy)

- **Ingress:** terminates HTTPS/gRPC from client applications.
- **PII/PHI Redaction Engine:** Microsoft Presidio + custom NER pipelines scrub data before transmission.
- **Policy Enforcement:** Open Policy Agent (OPA) validates traffic against enterprise rules.
- **Egress:** forwards sanitized payloads to external LLM providers, preserving native API compatibility.

4.2 Layer 2 — Compliance Engine & Agentic Chat

- **Orchestrator:** manages state and multi-step reasoning via LangGraph.
- **Policy-as-Code Engine:** YAML validator that keeps agents within authorized bounds.
- **Ephemeral Workers:** short-lived execution environments running AWS / GitHub / GCP connectors for scans and approved remediations.

4.3 Layer 3 — Storage & Monitoring

- **Metadata & Auth:** PostgreSQL for multi-tenant RBAC and application state.
- **Immutable Logs:** ClickHouse for high-volume, tamper-proof audit trails.

4.4 Technical Stack

Component	Technology
Gateway Proxy	Go / Rust (low latency)
Administrative APIs	Python (FastAPI)
Agent Framework	LangGraph / TypeScript workers
Relational Store	PostgreSQL (tenant config, RBAC, app state)
Audit Storage	ClickHouse (immutable hash-chained traces)
Message Broker	Apache Kafka
Console / Frontend	Next.js 15
Sensitive-data detection	Microsoft Presidio + custom NER
Policy engine	Open Policy Agent (OPA) + YAML policy-as-code
Secrets / KMS	AWS KMS or HashiCorp Vault (envelope AES-256-GCM)

5. Requirements Traceability

Every functional requirement is owned by an epic so nothing in the SRS is orphaned. Epic IDs are defined in Section 8.

Req. ID	Requirement	Owning epic
FR-1.1	Multi-model proxying (OpenAI, Anthropic, Cohere, Azure OpenAI), native payload compatibility	E1.4
FR-1.2	Real-time PII/PHI redaction — mask / SHA-256+salt hash / synthetic replacement	E2.1
FR-1.3	Policy enforcement (YAML + OPA), topic & regex blocking	E2.2
FR-2.1	Orchestrator–worker isolation with scoped, temporary tokens	E2.5
FR-2.2	Context-aware framework querying via RAG over regulatory docs	E2.4
FR-2.3	HITL workflow — PENDING_USER_APPROVAL, 0.5 h auto-expiry, MFA on execution	E2.6
FR-3.1	Automated framework scoring (SOC 2 / GDPR / HIPAA)	E3.2
FR-3.2	Cryptographically verified audit export	E4.4
NFR-1.1	≤ 50 ms gateway overhead per request	E4.3
NFR-1.2	Token-by-token streaming filtering, no fragmentation	E2.3
NFR-2.1	Tenant data isolation via RLS / physical isolation	E1.3
NFR-2.2	Envelope encryption (AES-256-GCM via KMS/Vault) for client credentials	E1.7
NFR-3.1	99.99% uptime, multi-region active-active	E4.5
NFR-3.2	Tiered rate limiting; throttle background scan workers	E1.4 / E4.5

6. HITL Action & Authorization Model

The agent must never execute a destructive change without an explicit, audited, expiring approval. This state machine is the contract for E2.6 and the approvals UI (E3.3).

State / trigger	Worker action	Required auth	Audit generated
Read-only scan	Fetch cloud config metadata	System account	Yes (Info)
Remediation plan	Generate Terraform / CLI diff	Orchestrator	Yes (Warning)
Execution	Apply configuration change	Human MFA approval	Yes (Critical)

Hard rule

Proposed destructive actions enter PENDING_USER_APPROVAL and auto-expire after 0.5 hours if unacted upon.

Execution requires a fresh MFA challenge bound to the specific action and approver; approvals are non-transferable and single-use.

7. Team, Roles & Capacity

7.1 Proposed Delivery Pod

Role	Count	Primary ownership
Tech Lead / Architect	1	Architecture, cross-cutting design, code review, security sign-off
Gateway Engineer (Go/Rust)	2	Proxy, redaction integration, streaming filter, latency
Backend Engineer (Python/FastAPI)	2	Control plane, orchestrator, workers, connectors, RAG
Frontend Engineer (Next.js 15)	1	Console, dashboard, chat & approvals UI
DevOps / SRE	1	IaC, CI/CD, ClickHouse/Kafka, multi-region, HA
QA / Security Engineer	1	Test automation, pentest coordination, red-team harness

7.2 Capacity Assumption (for the indicative calendar)

- Pod size: 8 engineers.
- Effective delivery capacity: ~32 productive hours/engineer/week → ~256 team-hours/week (the remainder absorbs ceremonies, review, and context-switching).
- At 256 team-hours/week, 640 hours ≈ ~2.5 weeks of build per phase; 2,560 hours ≈ ~10 weeks of build total.
- Calendar caveat: penetration testing and the SOC 2 observation window add elapsed time that is not counted as build hours (see Section 11).

7.3 RACI (high level)

Activity	Tech Lead	Gateway	Backend	Frontend	DevOps	QA/Sec
Architecture & design	A/R	C	C	C	C	C
Gateway & redaction	A	R	C	I	C	C
Agent engine & HITL	A	I	R	C	I	C
Console / UX	A	I	C	R	I	C
Infra, CI/CD, HA	A	C	C	I	R	C
Security & compliance	A	C	C	I	C	R

R = Responsible · A = Accountable · C = Consulted · I = Informed

8. Phased Work Breakdown (Epics & Tasks)

Four phases of 640 engineering hours each, mirroring the SRS roadmap. Each phase decomposes into epics whose hour budgets sum to 640. Within each epic, key tasks and acceptance criteria are listed.

8.1 Phase 1 — Foundation & Architecture

Budget: 640 engineering hours

Epic	Description	Hours	Owner
E1.1	Cloud foundation & IaC: Terraform modules, multi-region network scaffolding, VPC, KMS, secrets baseline.	96	DevOps
E1.2	CI/CD pipeline: build, unit-test, SAST/dependency scan, container build, environment promotion gates.	80	DevOps
E1.3	Multi-tenant data model: PostgreSQL schema, tenant model, row-level security (RLS), migrations.	96	Backend
E1.4	Gateway proxy skeleton: Go/Rust ingress, reverse-proxy for 4 providers, native payload passthrough, routing, base rate limiting.	160	Gateway
E1.5	Audit store: ClickHouse cluster, append-only audit schema, SHA-256 hash-chaining, write path.	96	Backend
E1.6	Event backbone: Apache Kafka topics, producer/consumer scaffolding for traffic & audit events.	64	DevOps
E1.7	Auth baseline: RBAC, OIDC/IdP integration, API-key service, envelope encryption of provider credentials.	48	Backend

Phase 1 — key tasks

- E1.4: implement provider-agnostic request/response adapters; verify OpenAI/Anthropic/Cohere/Azure payload fidelity with contract tests.
- E1.3: prove tenant isolation with an automated cross-tenant access test that must fail to read foreign rows.
- E1.5: design the hash-chain so each record's integrity_hash incorporates the prior record's hash (tamper-evidence).

Phase 1 — exit criteria

- A request flows client → gateway → provider → client through the proxy with payloads intact and an audit record written and chained.
- Two tenants are provably isolated at the database layer; CI/CD deploys to a staging environment automatically.

8.2 Phase 2 — Agentic Engine & Guardrails

Budget: 640 engineering hours

Epic	Description	Hours	Owner
E2.1	PII/PHI redaction engine: Presidio + custom NER; masking, SHA-256+salt hashing, synthetic replacement; reversible tokenization map per tenant.	140	Gateway
E2.2	Policy enforcement: OPA integration + YAML policy-as-code validator; topic-classification & regex blocking rules.	110	Backend
E2.3	Streaming filter: token-by-token chunk evaluation without stream fragmentation; back-pressure handling.	80	Gateway
E2.4	Orchestrator: LangGraph state machine, multi-step reasoning, RAG over GDPR/HIPAA/SOC 2 documentation.	120	Backend
E2.5	Ephemeral workers: short-lived runtimes, scoped temporary tokens, AWS/GCP/GitHub connectors for scans & remediation.	110	Backend
E2.6	HITL workflow engine: approval state machine, 0.5 h auto-expiry, MFA-gated execution, full audit emission.	80	Backend

Phase 2 — key tasks

- E2.1: support all three tokenization strategies and confirm redaction on both inbound prompts and outbound completions.
- E2.2: the YAML validator must reject malformed or over-scoped policies before they can route traffic.
- E2.5: tokens are scoped to the minimum capability and expire with the worker; no long-lived cloud credentials in workers.
- E2.6: implement the three-state model (read-only / plan / execute) from Section 6 exactly.

Phase 2 — exit criteria

- Sensitive data is redacted in real time including on streamed responses; policies block disallowed traffic before egress.
- The agent can scan, propose a Terraform/CLI diff, and apply it only after a fresh MFA-backed human approval; everything is audited.

8.3 Phase 3 — Developer Experience (Console)

Budget: 640 engineering hours

Epic	Description	Hours	Owner
E3.1	Console foundation: Next.js 15 app shell, auth, tenant context, design system, navigation per Section 3.	80	Frontend
E3.2	Compliance dashboard & framework scoring UI: live SOC 2 / GDPR / HIPAA readiness %, drill-downs.	120	Frontend
E3.3	Agent chat UI + remediation & HITL approvals queue with MFA confirmation flow.	120	Frontend
E3.4	Gateway & redaction config UI: routes, providers, per-route redaction strategy, live traffic inspector.	90	Frontend
E3.5	Audit log explorer + Trust Center: searchable immutable log, hash-chain verification badge, shareable trust page.	110	Frontend
E3.6	Tenant admin: RBAC management, users, API-key lifecycle UI, rate-limit tiers.	70	Frontend
E3.7	Public API + SDK + developer docs for gateway onboarding and audit export.	50	Backend

Phase 3 — exit criteria

- An admin can configure a gateway route, watch redaction happen live, read framework scores, and approve an agent action entirely from the console.
- Audit records are browsable and individually verifiable against the hash chain from the UI.

8.4 Phase 4 — Compliance Hardening

Budget: 640 engineering hours

Epic	Description	Hours	Owner
E4.1	Penetration testing & remediation: external pentest, threat-model validation, fix cycle.	140	QA/Sec
E4.2	Continuous red-teaming harness: adversarial probes (prompt injection, data disclosure, sycophancy, harmful content) + vulnerability register.	120	QA/Sec
E4.3	Latency optimization: profile & tune to ≤ 50 ms gateway overhead; load testing at target throughput.	110	Gateway
E4.4	Cryptographic audit export: signed, verifiable compliance reports with logs, redaction metrics, config snapshots.	90	Backend
E4.5	HA & resilience: multi-region active-active, failover, chaos testing toward the 99.99% target.	110	DevOps
E4.6	SOC 2 evidence automation + documentation: control evidence pipeline, runbooks, policies.	70	QA/Sec

Phase 4 — exit criteria

- Gateway overhead is measured at ≤ 50 ms under load; red-team probes pass thresholds; audit exports verify cryptographically.
- Active-active failover is demonstrated; SOC 2 evidence is collected automatically.

8.5 Effort Roll-up

Phase	Focus	Hours
Phase 1	Foundation & Architecture	640
Phase 2	Agentic Engine & Guardrails	640
Phase 3	Developer Experience (Console)	640
Phase 4	Compliance Hardening	640
Total	MVP build	2,560

9. Sprint Plan

Two-week sprints at ~512 team-hours each (256 team-hours/week). Five sprints cover the 2,560-hour MVP build. Sprints are sequenced to respect dependencies — foundation before engine, engine before console surfaces that depend on it, hardening last.

Sprint	Team-hours	Primary epics	Sprint goal
S1	512	E1.1, E1.2, E1.3, E1.4 (start)	Infra, CI/CD, tenant model, proxy passthrough working in staging.
S2	512	E1.4 (finish), E1.5, E1.6, E1.7, E2.1 (start)	Audit chain + event backbone live; redaction engine begun.
S3	512	E2.1 (finish), E2.2, E2.3, E2.4 (start)	Real-time + streaming redaction and policy enforcement in place.
S4	512	E2.4 (finish), E2.5, E2.6, E3.1, E3.2 (start)	Agent orchestrator, workers, HITL; console foundation + scoring begun.
S5	512	E3.2–E3.7	Full console (chat, approvals, gateway config, audit explorer, admin, API).
S6–S7	≈1,280	E4.1–E4.6	Hardening: pentest, red-team, latency, crypto export, HA, SOC 2 evidence.

Note: Phase 4 spans roughly two sprints of build plus the external pentest and SOC 2 observation windows, which add calendar time beyond the build hours.

10. Environments, DevOps & Data

10.1 Environments

Env	Purpose	Notes
Dev	Per-engineer / feature work	Ephemeral, seeded test tenants
Staging	Integration & pre-prod validation	Mirror of prod topology, synthetic traffic
Prod	Multi-region active-active	99.99% target, strict change control

10.2 CI/CD & Quality Gates

- Pipeline stages: lint → unit tests → SAST + dependency scan → container build → integration tests → deploy (gated promotion).
- No deploy to staging/prod without green tests and a passing security scan.
- Infrastructure is provisioned only through Terraform; no manual console changes in prod.

10.3 Core Audit Record (ClickHouse)

The tamper-proof audit record is the spine of the compliance story. Fields per the SRS schema:

Field	Type / format	Notes
record_id	uuid	Unique per record
tenant_id	uuid	Tenant scope
timestamp	date-time	Event time
actor	object {id, type}	type ∈ user / agent / system_gateway

Field	Type / format	Notes
action	string	What occurred
frameworks_affected	array<string>	GDPR / HIPAA / SOC2
execution_trace	array<step>	step, component, rationale, output_summary
integrity_hash	string	SHA-256 chain hash (links to prior record)

11. Testing & Validation Strategy

Test type	What it covers	Owner / phase
Unit	Adapters, redaction strategies, policy validation, HITL state transitions.	All / continuous
Contract	Native payload fidelity per provider (OpenAI, Anthropic, Cohere, Azure).	Gateway / P1–P2
Integration	End-to-end proxy → redact → policy → egress → audit chain.	QA / P2–P3
Multi-tenant isolation	Cross-tenant access must fail; RLS enforced.	QA / P1
Streaming	Token-by-token filtering without fragmentation under load.	Gateway / P2
Load & latency	≤ 50 ms gateway overhead at target throughput.	Gateway / P4
Red teaming	Prompt injection, data disclosure, sycophancy, harmful-content probes.	QA/Sec / P4
Penetration	External pentest of gateway, auth, tenant boundaries.	QA/Sec / P4
Audit integrity	Hash-chain verification; export validates cryptographically.	Backend / P4
HA / chaos	Region failover; resilience toward 99.99%.	DevOps / P4

12. Risk Register

#	Risk	Impact	Likelihood	Mitigation
R1	Redaction adds latency that breaks the 50 ms budget.	High	Med	Profile early in P2; choose Go/Rust hot path; defer heavy NER to async where safe; load test in P4.
R2	Streaming filter fragments responses or drops tokens.	High	Med	Prototype chunk evaluation in S3; back-pressure tests; provider-specific stream tests.
R3	Over-scoped worker tokens enable lateral movement.	High	Low	Minimum-capability scoping, short TTL, per-action tokens, audit every grant.
R4	Tenant isolation gap leaks data across tenants.	Critical	Low	RLS + automated cross-tenant tests in P1; pentest in P4.
R5	HITL bypass lets agent execute without approval.	Critical	Low	Single, audited execution path gated by fresh MFA; deny-by-default; expiry enforced server-side.
R6	Audit log tampering undermines compliance claims.	Critical	Low	Append-only ClickHouse, SHA-256 chaining, verification tooling, restricted write path.
R7	Provider API drift breaks payload compatibility.	Med	Med	Contract tests in CI; adapter layer isolates provider changes.
R8	Pentest / SOC 2 windows extend the calendar.	Med	High	Start evidence automation in P4 early; book pentest ahead; treat as elapsed-time, not build-hours.

13. Definition of Done & Dependencies

13.1 Definition of Done (per story)

- Code reviewed and merged; unit + relevant integration tests passing in CI.
- SAST/dependency scan clean; no new high/critical findings.
- Telemetry and audit emission wired where applicable.
- Tenant-scoped and authorization-checked; no cross-tenant leakage.
- Documented (API/user) and demoable in staging.

13.2 Definition of Done (per phase)

- All epic exit criteria met; requirement IDs in Section 5 for that phase verified.
- No open critical/high defects; risk register reviewed and updated.

13.3 Key Dependencies & Assumptions

- Cloud accounts (AWS primary per stack; GCP/GitHub for connectors) provisioned before S1.
- LLM provider API keys for OpenAI, Anthropic, Cohere, Azure OpenAI available for contract testing.
- Access to regulatory documentation corpus (GDPR / HIPAA / SOC 2) for the RAG index.
- External pentest vendor and SOC 2 auditor engaged ahead of Phase 4.
- Estimates assume the 8-person pod and capacity in Section 7; changes rescale the calendar, not the hour totals.

14. Milestones (by cumulative build hours)

Milestone	Marker	Cumulative hours
M1 — Proxy passthrough + tenant isolation	End of Phase 1	640
M2 — Redaction, policy & agentic engine	End of Phase 2	1,280
M3 — Full admin console	End of Phase 3	1,920
M4 — Hardened, audit-ready MVP	End of Phase 4	2,560

Prepared by Binod Kumar · AgentsArchitects.ai
<https://agentsarchitects.ai/>